

Centro Universitário UniFECAF

Cloud DevOps: Orchestrating Containers and Micro Services

KEVIN MAIKON CAETANO DE ANDRADE SANTOS

**Entrega contínua de uma plataforma de pedidos em microsserviços: do
Docker Compose ao Kubernetes com observabilidade e CI/CD**

Relatório técnico prático

Taboão da Serra
2026

1 INTRODUÇÃO

Este relatório técnico prático descreve a implementação do MVP Pedidos Veloz para a Loja Veloz. A entrega cobre ambiente local com Docker Compose, containerização e versionamento de imagens, manifests Kubernetes, pipeline de CI/CD, observabilidade, estratégia de *deploy* e escala, além de infraestrutura como código com Terraform. O código-fonte encontra-se no repositório do projeto.

2 AMBIENTE LOCAL COM DOCKER COMPOSE

O arquivo *docker-compose.yml* sobe os serviços postgres, estoque, pagamentos, pedidos e api-gateway em uma rede *bridge* dedicada, com volume persistente para o banco e *script* de inicialização em *scripts/init-db.sql*. As variáveis de ambiente são carregadas a partir do arquivo *.env*, tendo *.env.example* como modelo versionado.

Com o comando *docker compose up --build*, toda a plataforma sobe de forma reproduzível. O API Gateway expõe a porta 8080. O fluxo principal é: o cliente envia POST */api/pedidos* ao gateway; o serviço de pedidos reserva estoque, solicita autorização de pagamento e confirma ou libera os itens. As respostas incluem identificador de *trace* para correlação.

3 CONTEINERIZAÇÃO E VERSIONAMENTO

Cada microsserviço possui Dockerfile multi-stage: a etapa *deps* instala dependências de produção; a etapa *runner* utiliza *node:20-alpine*, executa como usuário não-root, inclui HEALTHCHECK e mantém apenas os artefatos necessários. As imagens são tagueadas como *lojaveloz/<servico>:1.0.0*. Credenciais não entram na imagem; são injetadas por ambiente, ConfigMap e Secret.

4 KUBERNETES — PRODUÇÃO MÍNIMA

Os manifests em *k8s/* são aplicados com *kubectl apply -k k8s/*. O Namespace utiliza Pod Security Admission (*enforce=baseline*). Há ConfigMap, Secret, Deployments com RollingUpdate, Services, *probes* HTTP, *requests/limits*, HorizontalPodAutoscaler e NetworkPolicies. A estratégia de *deploy* é RollingUpdate com *maxUnavailable* igual a zero e *maxSurge* igual a um. O HPA escala o api-gateway (2 a 8 réplicas) e o serviço de pedidos (2 a 10 réplicas) com base na utilização de CPU.

5 CI/CD

O pipeline em *.github/workflows/ci-cd.yml* executa testes e verificação de sintaxe por serviço, *build* e *push* das imagens para o GHCR, scan com Trivy e *gate* de ambiente *staging* com validação dos manifests. O GITHUB_TOKEN autentica a publicação das imagens.

6 OBSERVABILIDADE E TERRAFORM

Os serviços emitem logs JSON em *stdout*. O middleware compartilha o cabeçalho W3C *traceparent* entre gateway, pedidos, estoque e pagamentos. O OpenTelemetry Collector está em *observability/*. O diretório *terraform/* define provider AWS, variáveis, *tags*, estrutura para módulos VPC/EKS e backend remoto S3/DynamoDB.

7 EXECUÇÃO, VALIDAÇÃO E RESULTADOS

Ambiente local: *cp .env.example .env && docker compose up --build*. Testes: *npm run install:all && npm test*. Kubernetes: *kubectl apply -k k8s/*. A validação funcional confirmou o *health* do gateway, a listagem de estoque e a criação de pedido com status CONFIRMADO, incluindo valor total, identificador de pagamento e *traceId*. Evidências detalhadas constam no anexo *Evidencias_Pedidos_Veloz.pdf* e em *docs/evidencias/*.

Como referência de mercado, adotou-se o Google Online Boutique (microservices-demo), complementado pelas documentações oficiais das tecnologias utilizadas.